

C++ Revision Guide

By Kuldeep Parmar • Topic-wise explained with code examples

1. Data Types and Variables

■ A variable is a named storage location in memory. A data type tells the compiler what kind of data the variable will hold.

Data Type	Size	Example	What it stores
int	4 bytes	int a = 10;	Whole numbers
float	4 bytes	float b = 3.14f;	Decimal (~6 digits)
double	8 bytes	double c = 3.14159;	Decimal (~15 digits)
char	1 byte	char d = 'k';	Single character
bool	1 byte	bool e = true;	true (1) or false (0)

■ *sizeof() operator returns how many bytes a type occupies: sizeof(int) → 4*

2. Input / Output

■ cout (Character OUTput) prints data to the screen using '<<'. cin (Character INput) reads data from the keyboard using '>>'.
■ endl flushes the buffer and moves the cursor to the next line. '\n' also moves to next line but is slightly faster.

```
cout << "Enter age: " << endl;  
cin >> age;
```

3. Control Flow Statements

3.1 if / else if / else

■ Used for decision-making. The block whose condition is true gets executed.

```
if(age >= 18)      { cout << "Adult"; }  
else if(age >= 13) { cout << "Teen";  }  
else              { cout << "Child"; }
```

3.2 Nested if

■ An if statement written inside another if. Used to check multiple related conditions.

```
if(age >= 18) {  
    if(age >= 60) cout << "Senior citizen";  
    else        cout << "Adult";  
}
```

3.3 Switch-Case

■ Compares a variable to a list of constant values. 'break' stops execution after a match. 'default' runs when nothing matches.

```
switch(day) {  
    case 1: cout << "Monday"; break;
```

```

    case 2: cout << "Tuesday"; break;
    default: cout << "Invalid";
}

```

3.4 Ternary Operator

■ A one-line shorthand for if-else. Syntax: condition ? valueIfTrue : valueIfFalse

```
(age >= 18) ? cout << "Eligible" : cout << "Not eligible";
```

4. Loops

■ Loops repeat a block of code. Use for-loop when the count is known, while-loop when the count depends on a condition.

4.1 for loop

```

for(int i = 1; i <= 5; i++) {
    cout << i << endl;
}

```

4.2 while loop

```

int i = 1;
while(i <= 5) {
    cout << i << endl;
    i++;
}

```

4.3 do-while loop

■ Executes the body AT LEAST ONCE before checking the condition.

```

int i = 1;
do {
    cout << i << endl;
    i++;
} while(i <= 5);

```

4.4 continue and break

■ continue skips to the next iteration. break exits the loop immediately.

```

for(int i = 1; i <= 10; i++) {
    if(i % 2 == 0) continue; // skip even
    if(i == 7) break;       // stop at 7
    cout << i;
}

```

5. Operators

5.1 Unary Operators

■ Operate on a single variable. Post-increment (x++) uses the value first then adds 1. Pre-increment (++x) adds 1 first then uses the value.

Operator	Name	Example	Result (x=5)
----------	------	---------	--------------

x++	Post-increment	cout << x++;	prints 5, x becomes 6
++x	Pre-increment	cout << ++x;	x becomes 6, prints 6
x--	Post-decrement	cout << x--;	prints 5, x becomes 4
--x	Pre-decrement	cout << --x;	x becomes 4, prints 4

5.2 Arithmetic Operators

```
int a=10, b=3;
a+b=13  a-b=7  a*b=30  a/b=3 (integer division)  a%b=1
```

■ *Integer division drops the remainder. 10/3 = 3, not 3.33*

5.3 Relational Operators

■ Compare two values. Return 1 (true) or 0 (false). == != > < >= <=

5.4 Logical Operators

Operator	Meaning	True when
&&	AND	Both sides are true
	OR	At least one side is true
!	NOT	Reverses true↔false

5.5 Bitwise Operators

■ Operate on binary (bit-level) representation of integers.

```
int m=5 (0101), n=3 (0011)
m & n = 0001 = 1
m | n = 0111 = 7
m ^ n = 0110 = 6
~m    = -6
m<<1  = 1010 = 10 (multiply by 2)
m>>1  = 0010 = 2  (divide by 2)
```

6. Number Systems

■ Computers store everything in binary (base-2). C++ lets you write literals in different bases.

System	Base	Prefix	Example	Decimal Value
Decimal	10	none	10	10
Binary	2	0b	0b1010	10
Octal	8	0	012	10
Hexadecimal	16	0x	0xA	10

■ *Binary→Decimal: multiply each bit by its place value (2^{position}) from right and add all.*

7. Type Casting

■ Converting a value from one data type to another.

Implicit (automatic)

■ Compiler converts automatically when types are compatible. Lower type promotes to higher type.

```
int i = 10;
float f = 1.5f;
float result = i + f; // int 10 → float 10.0, result = 11.5
```

Explicit (manual)

■ Programmer forces the conversion. Decimal part is TRUNCATED (cut off), not rounded.

```
float b = 3.99f;
int a = (int)b; // a = 3 (not 4)
```

8. Functions

■ A function is a reusable named block of code that performs a specific task. Functions reduce repetition and make code modular.

```
// Syntax
returnType functionName(parameters) {
    // body
    return value;
}

// Example
int sum(int x, int y) {
    return x + y;
}
```

■ If defined BEFORE main(), no prototype needed. If defined AFTER main(), add a prototype before main(): `int sum(int, int);`

Pass by Value vs Pass by Reference

	Pass by Value	Pass by Reference
Syntax	<code>void fn(int x)</code>	<code>void fn(int &x)</code>
What is passed	A copy of the variable	The actual variable
Changes affect original?	No	Yes
Use when	You don't want to modify original	You want to modify original

9. Arrays

■ An array is a collection of elements of the SAME data type stored in consecutive memory locations. Index starts at 0.

```
int arr[5] = {1, 2, 3, 4, 5};
// arr[0]=1, arr[1]=2, ..., arr[4]=5

for(int i = 0; i < 5; i++)
    cout << arr[i];
```

■ **Bug Fix:** Original code used 'index <= 5' which reads arr[5] — one slot beyond the array (undefined behavior). Corrected to 'index < 5'.

10. Strings

Char Array (C-style)

■ A character array ending with '\0' (null terminator). cin>> reads only one word.

```
char name[20];
cin >> name; // reads until space
```

std::string (C++ class)

■ The string class from library is much easier to use and has many useful methods.

Method	What it does	Example
.length()	Returns number of characters	str.length() → 13
.append(s)	Adds s to the end	str.append(" hi")
.insert(p, s)	Inserts s at position p	str.insert(5, " Mr.")
.erase(p, n)	Removes n characters from pos p	str.erase(5, 4)
.replace(p, n, s)	Replaces n chars at pos p with s	str.replace(0,5,"Hi")

11. Pointers and Dynamic Memory

■ A pointer is a variable that stores the MEMORY ADDRESS of another variable.

```
int number = 10;
int *ptr = &number; // ptr stores address of number

cout << number; // 10 (value)
cout << &number; // 0x7ffd.. (address)
cout << ptr; // 0x7ffd.. (same address)
cout << *ptr; // 10 (value at that address)
```

■ *& = address-of operator * = dereference operator (get value from address)*

Dynamic Memory Allocation

■ 'new' allocates memory on the HEAP at runtime. Always use 'delete' to free it and avoid memory leaks.

```
int *ptr2 = new int; // allocate
*ptr2 = 20;
cout << *ptr2; // 20
delete ptr2; // free memory
```

12. Classes and Objects

■ A class is a blueprint for creating objects. It bundles data (attributes) and functions (methods) together. An object is a specific instance of a class.

```
class Person {
public:
    string name;
    int age;
    void display() {
        cout << name << " " << age;
    }
};

// Creating an object
Person p1;
p1.name = "Kuldeep";
```

```
p1.age = 25;
p1.display();
```

Constructor

■ A special method called automatically when an object is CREATED. It has the same name as the class and no return type. Used to set initial values.

```
Person() { // Default constructor
    name = "Unknown";
    age = 0;
    cout << "Object created";
}
```

Destructor

■ Called automatically when an object is DESTROYED (goes out of scope). Prefix ~. Used to release resources.

```
~Person() {
    cout << "Object destroyed: " << name;
}
```

■ *Destructors are called in REVERSE order of construction when the program ends or object goes out of scope.*

13. Quick Reference — Key Concepts at a Glance

Concept	One-line definition
Variable	Named storage location in memory
Data Type	Tells compiler what kind of data to store
if-else	Run code based on a condition
switch	Compare variable to multiple constant values
for loop	Repeat code a known number of times
while loop	Repeat code while condition is true
Function	Reusable named block of code
Pass by Value	Copy passed; original unaffected
Pass by Reference	Actual variable passed; changes affect original
Array	Fixed-size collection of same-type elements
Pointer	Variable that stores a memory address
new / delete	Allocate / free heap memory at runtime
Class	Blueprint for creating objects
Constructor	Auto-called method when object is created
Destructor	Auto-called method when object is destroyed
Type Casting	Converting one data type to another
Bitwise operators	Operate on binary representation of numbers